

1. The Problem — Drug Discovery Is a \$2.5B, 12-Year False-Positive Machine

The market problem and the scientist problem are the same root issue: biology is hard to predict, expensive to validate, and poorly connected to learning systems.



Market / Industry Problem



\$2.5B average cost and ~10–12 years to bring a drug to market



Most candidates fail late because early signals create false positives



Biology is nonlinear, noisy, and much less predictable than software systems



Each failed candidate wastes wetlab time, capital, and program momentum



\$2.5B

avg. development cost



~12 yrs

typical timeline



Scientist / User Problem



Literature is fragmented across thousands of papers



Experimental data is messy, multi-format, and hard to compare



Hypothesis generation is slow and manually intensive



Candidate ranking is inconsistent across tools and teams



Lab results rarely flow back into models in a reusable way



What's Missing



Biological evidence



Candidate ranking



Lab validation



Model improvement



No unified closed loop from evidence to experiment to learning



Human teams stitch together disconnected tools and spreadsheets



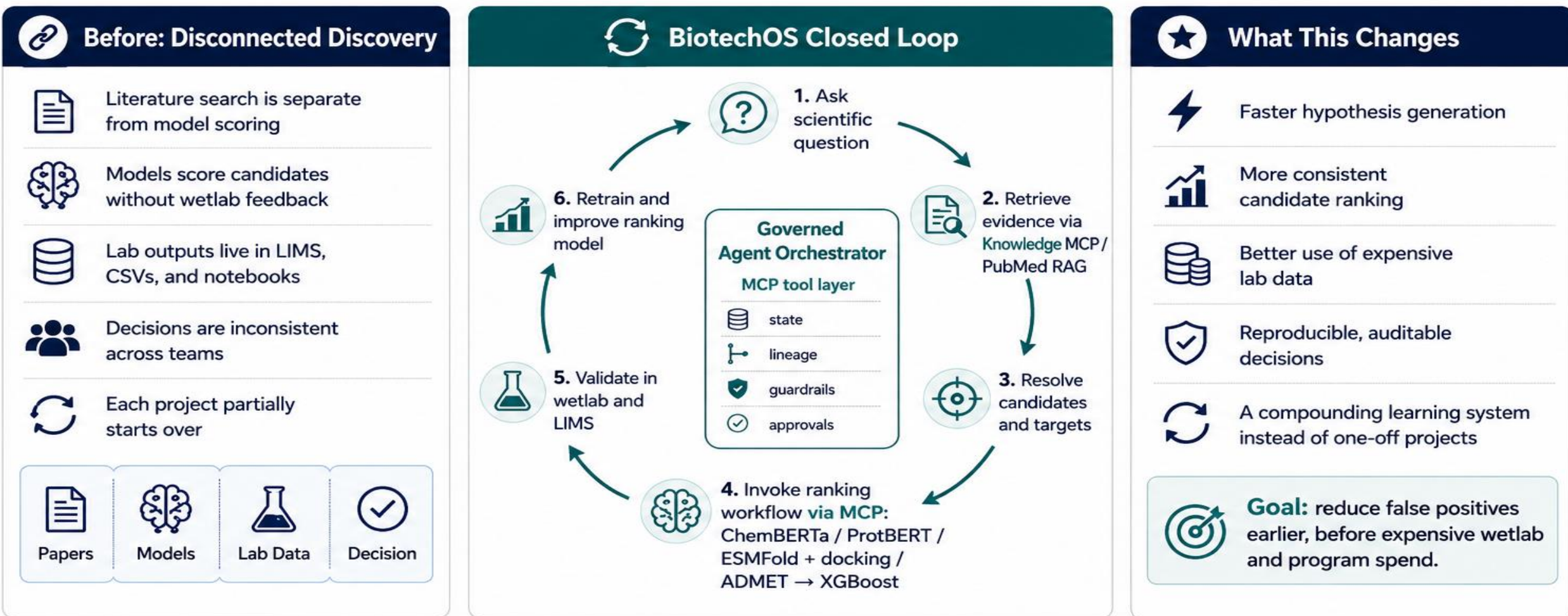
Every project partially starts over



Core insight: the problem is not just search or prediction — it is the absence of a governed learning loop that connects literature, models, lab data, and scientific decisions.

2. The Product Thesis — Turn Drug Discovery Into a Governed Learning System

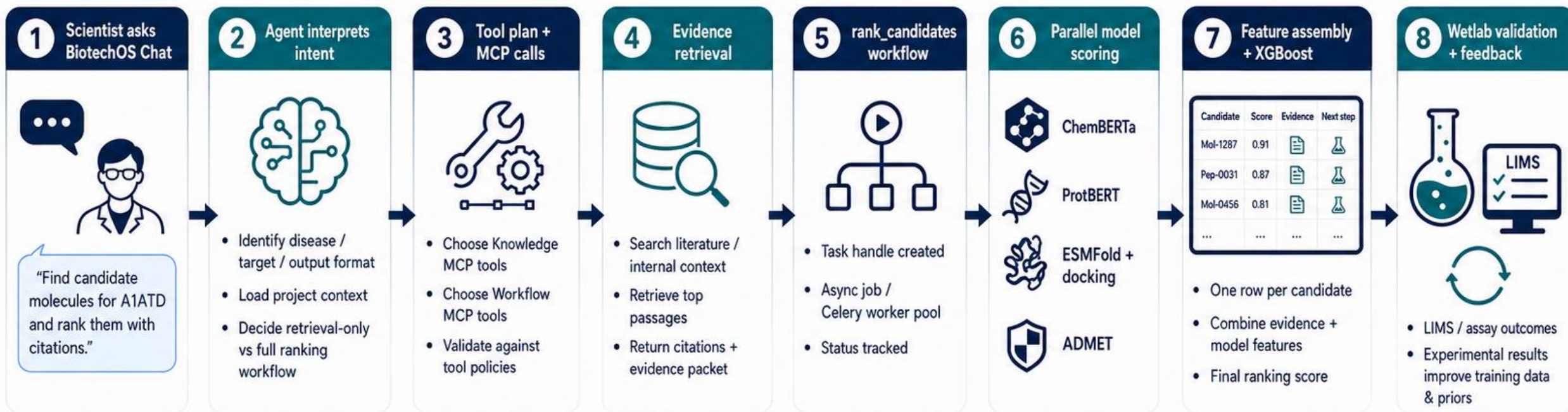
BiotechOS connects literature, multimodal models, lab validation, and retraining into one reusable decision loop.



Core insight: the product is not just a chat interface or a model — it is a governed learning loop that turns biological evidence into compounding prediction quality.

3. End-to-End User Workflow — From Question to Ranked Candidates

How a scientist uses BiotechOS in practice when an agent plans, calls tools through MCP, and launches governed workflows.



What the scientist sees

- Ranked shortlist
- Citations
- Rationale
- Next experiment



What the system does behind the scenes

- Agent orchestration across retrieval, models, and tools
- MCP-based tool access with schema validation & policy checks
- Async workflow launch, feature assembly & scoring
- Audit trail for transparency & reproducibility



Why it matters

- Less manual stitching
- Faster iteration
- Reusable learning

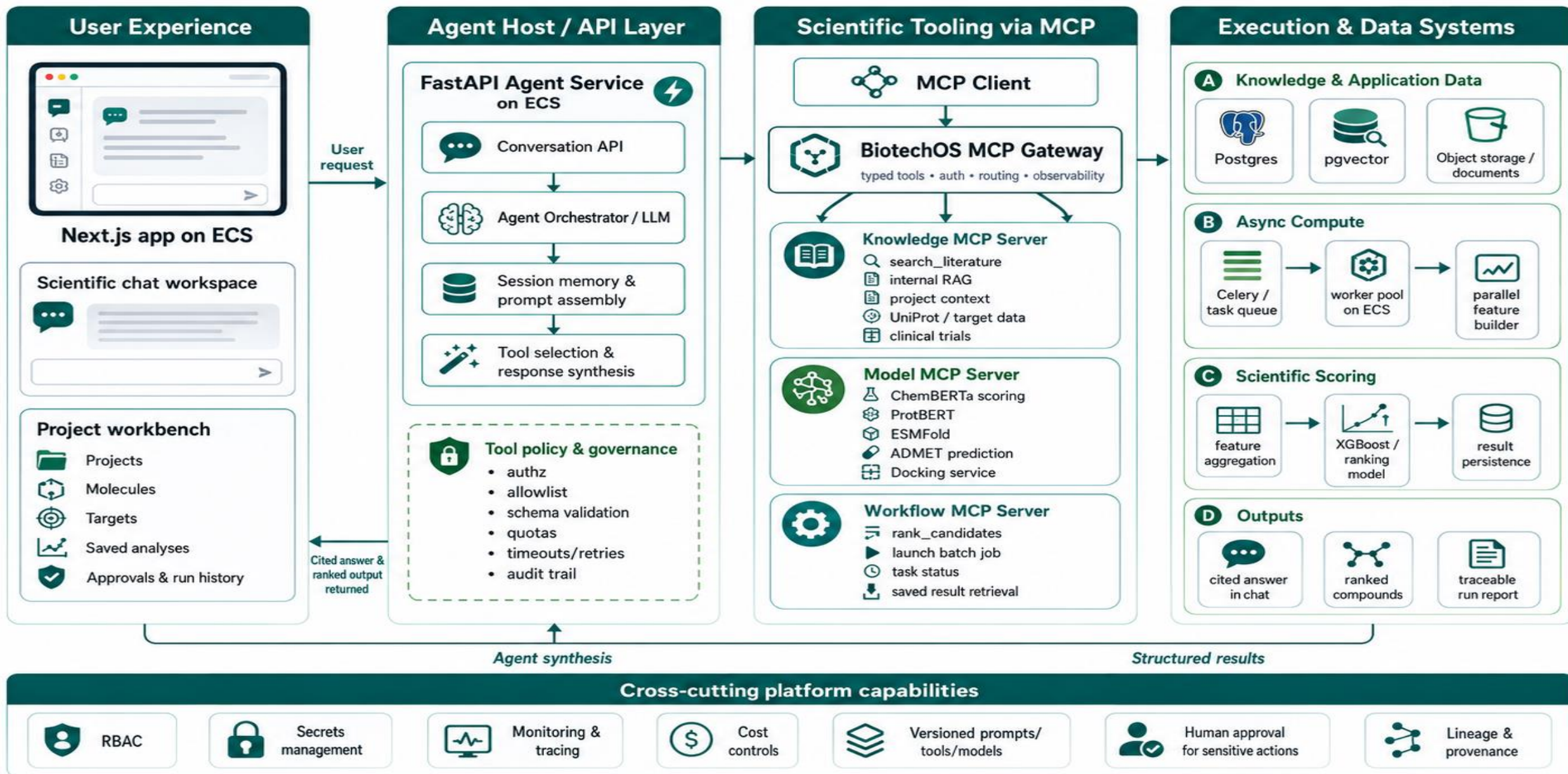


Core takeaway: the user experience is simple; the technical work is in turning one prompt into a governed multi-tool, agentic workflow.



BiotechOS Agentic Scientific Copilot Architecture

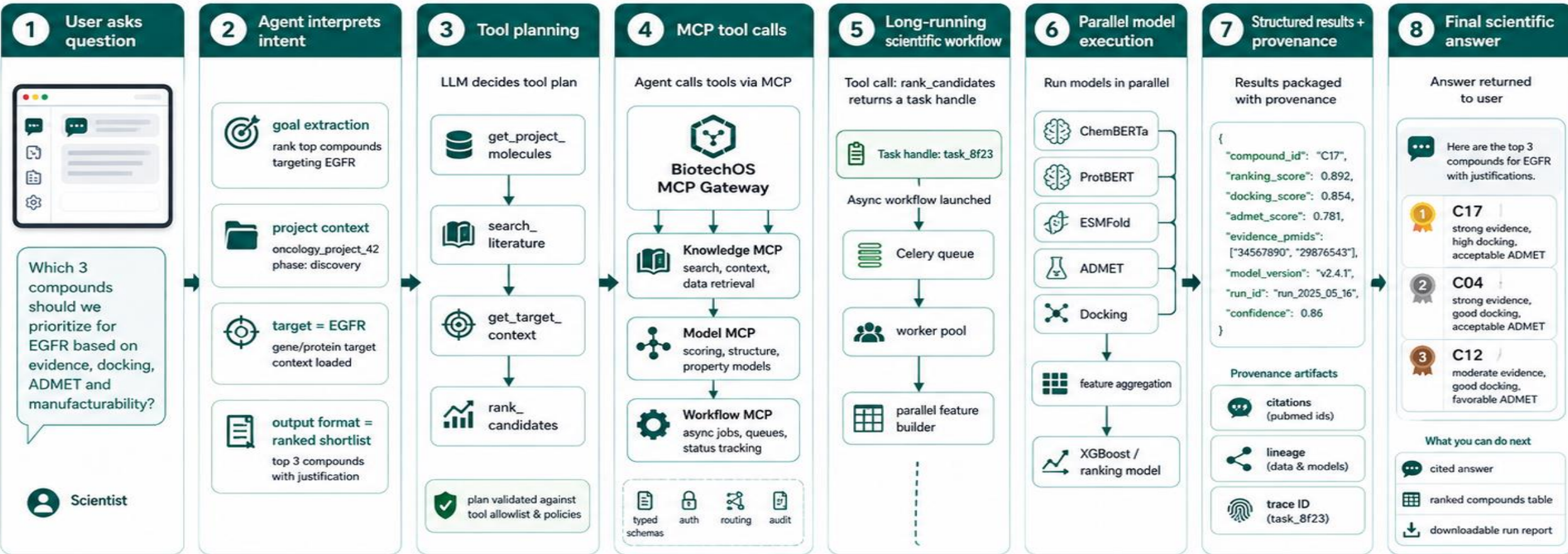
Next.js frontend + FastAPI agent backend on AWS ECS with tool-calling, MCP, scientific models, and governed workflows





BiotechOS Scientific Agent Request Flow

How the copilot plans, calls tools through MCP, runs async scientific workflows, and returns traceable answers.



Built-in control points



RBAC



tool allowlist



quota limits



human approval for expensive actions



monitoring



versioned models/prompts



full audit trail

BiotechOS AI Core – RAG in Practice

How the BiotechOS agent uses the Knowledge MCP Server to build evidence from PubMed and answer scientist questions with pgvector-based RAG.

1) TRAINING / INGESTION (Offline Knowledge Build)

1 PubMed / PMC → Chunks + Embeddings



Ingestion & Chunking

- Fetch articles via PubMed / PMC
- Parse & clean text
- Chunk into passages
- Embed with MedCPT
- Store in pgvector



Example Stored Vector Record

```
{
  "chunk_id": "PMID123_chunk_57",
  "text": "Tumor-infiltrating macrophages...",
  "embedding": [0.001, -0.024, ...],
  "metadata": {
    "pmid": "123",
    "title": "...",
    "year": 2024,
    "journal": "Nat Immunol",
    "section": "Results"
  }
}
```

pgvector Vector Store —
semantic search over PubMed chunks / passages
with metadata and citations.

2 Chunks + Metadata



Documents → Chunks → Embeddings



Store chunk text, embeddings,
and metadata.
(PMID, title, year, journal, section, etc.)

3 Citation-aware RAG Index

- ✓ Chunks (text passages)
- ✓ Embeddings (vectors)
- ✓ Metadata & citations

- PMID
- Title
- Year
- Journal
- Section

Built-in ANN index
for fast semantic search



2) QUERY / INFERENCE (Answering Scientist Questions)

1 User Query

"Find molecules for A1ATD"



2 Agent selects Knowledge MCP tool



search_literature() /
Knowledge MCP Server

Agent invokes the
Knowledge MCP Server
to search the literature.

3 Query Embedding + Vector Search (pgvector)



LlamaIndex
Query Engine
Embed query
with MedCPT

Search pgvector
(ANN Search)

Example Output

```
{
  "query_embedding": "[...]",
  "index": "pgvector",
  "k": 50
}
```

4 Retrieve Top-K Similar Chunks

Top-K results from pgvector

Example Output (Retrieved Chunks)

```
{ "pmid": "123", "text": 0.91, "score": "Tumor-infiltrating macrophages...",
{ "pmid": "456", "text": 0.87, "score": "METHODS: Inoculation of..."},
{ "pmid": "789", "text": 0.82, "score": "Discussion of the role of A1ATD..." }
```

5 Re-rank / Filter Evidence



Optional Re-ranking
MedCPT cross-encoder
(or other reranker)

Metadata Filtering
(year, journal, section,
disease, etc.)

Example Output (Re-ranked)

```
{ "pmid": "123", "score": 0.95, "text": "Tumor-infiltrating macrophages...",
{ "pmid": "799", "score": 0.87, "text": "METHODS: Inoculation of..."},
{ "pmid": "456", "score": 0.84, "text": "Discussion of the role of A1ATD..." }
```

6 Structured Evidence Packet returned to Agent

Evidence Packet (to Agent)

- Top K re-ranked passages (text)
- Metadata & citations (PMID, title, year, journal)
- Retrieval scores

Example Output (JSON)

```
{
  "evidence": [
    { "pmid": "123", "score": 0.95, "text": "...", "year": 2024, "journal": "Nat Immunol" },
    { "pmid": "789", "score": 0.89, "text": "...", "year": 2023, "journal": "Transl Med." }
  ],
  "query": { "A1ATD": 50, "retrieved": 50, "reranked": 10 }
}
```

Agent synthesis after RAG

- ✓ Ranked candidates
- ✓ Mechanism of action
- ✓ Evidence (PMIDs)
- ✓ Confidence score



Key takeaway: RAG retrieves the most relevant PubMed passages and metadata, exposes them through the Knowledge MCP Server, and provides the agent with the evidence needed to synthesize the final answer.

Model I/O Deep Dive — What Each Model Actually Took In and Produced

Interview-safe truth: foundation models produced features or structure; BiotechOS assembled those outputs into one disease-specific scoring row.

1 PubMed RAG / Graph RAG



- Input:** scientist question + disease / target query
- Runtime:** LlamaIndex + vector DB + structured extraction
- Output:** evidence JSON with fields such as disease, target, mechanism, candidate_molecule, evidence_score, PMIDs

```
{
  "disease": "Alzheimer's disease",
  "target": "BACE1",
  "mechanism": "inhibition",
  "candidate_molecule": "CHEMBL12345",
  "evidence_score": 0.72,
  "PMIDs": ["34567800", "29076543", "31524600"]
}
```

2 ChemBERTa / ProtBERT



ChemBERTa

- Input format:** SMILES
- Example:** CC[=O]OC1...
- Runtime:** Hugging Face / transformers
- Output:** molecule embedding (e.g., 768-d)

[0.12, -0.31, 0.47, ..., -0.88] (766-d)

ProtBERT

- Input format:** amino-acid sequence
- Example:** MKTIIALSYIPCLVFA...
- Runtime:** Hugging Face / transformers
- Output:** protein embedding (e.g., 1024-d)

[-0.21, 0.06, 0.32, ..., 0.17] (1024-d)

3 ESMFold + Docking



- Input:** target protein amino-acid sequence
- ESMFold output:** PDB structure + pLDDT confidence

structure.pdb
mean_pLDDT: 0.78

- Docking step:** ligand + receptor structure -> AutoDock Vina -> docking score in kcal/mol

docking_score: -8.6 kcal/mol

i ESMFold does NOT output docking score by itself

4 ADMET model



- Input:** SMILES
- Output:** property predictions such as bioavailability, hERG risk, CYP inhibition, solubility, clearance

Property	Predicted value
Bioavailability (P%)	0.43
hERG risk (pICE0)	-5.21
CYP3A4 inhibition (prob)	0.18
CYP2D6 inhibition (prob)	0.07
Solubility (logS)	-3.12
Clearance (mL/min/kg)	12.6

5 Feature assembly + XGBoost



BiotechOS builds one numeric row per candidate with columns like:

evidence_score	chemberts_emb_1	chemberts_emb_2	...	docking_score	pLDDT	HERG_risk	ganefarmes_risk (optional)	label_y (stat) / score (nIw)
0.72	0.12	-0.31	...	-0.6	0.78	-5.21	0.44	1



XGBoost

- predict_proba(X)** -> efficacy probability
- predict(X)** -> rescue / ranking score

★ XGBoost learned from wetlab labels; the pretrained models were feature generators, not final decision makers.



Input formats

- SMILES (chemical structure)
- Amino-acid sequence (protein)
- Target sequence (for structure prediction)
- Question text (scientist query)



Outputs

- Embeddings (molecule & protein)
- Structure (PDB) + pLDDT
- ADMET property predictions
- Evidence JSON (from RAG / Graph RAG)



Why this mattered

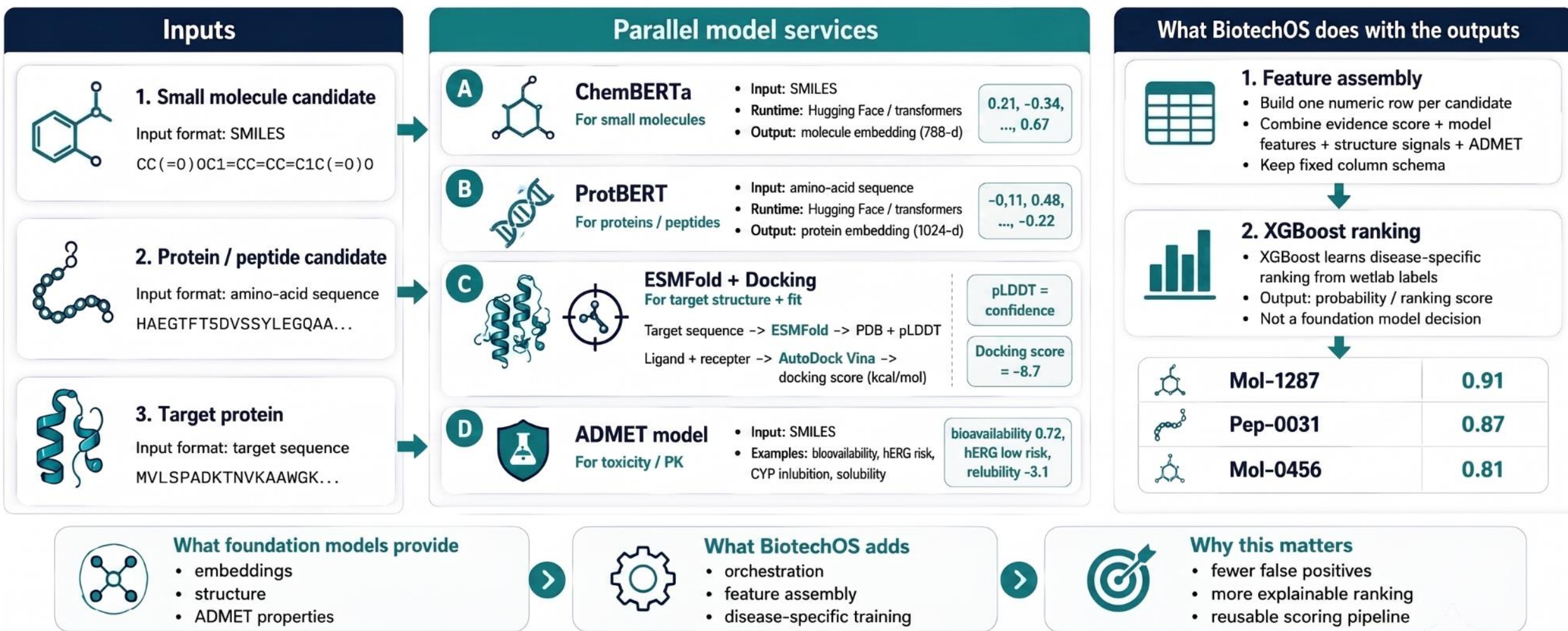
- Explainable: every number has a source and meaning
- Modular: clear input/output contracts between components
- Easy to swap: upgrade or replace any model without breaking the pipeline
- Disease-specific ranking: XGBoost learns from wetlab labels for that disease



Core takeaway: BiotechOS was credible because every model had a concrete input, a concrete output, and a defined role in the final decision path.

Multimodal Model Pipeline — How BiotechOS Turns Candidates into Ranked Signals

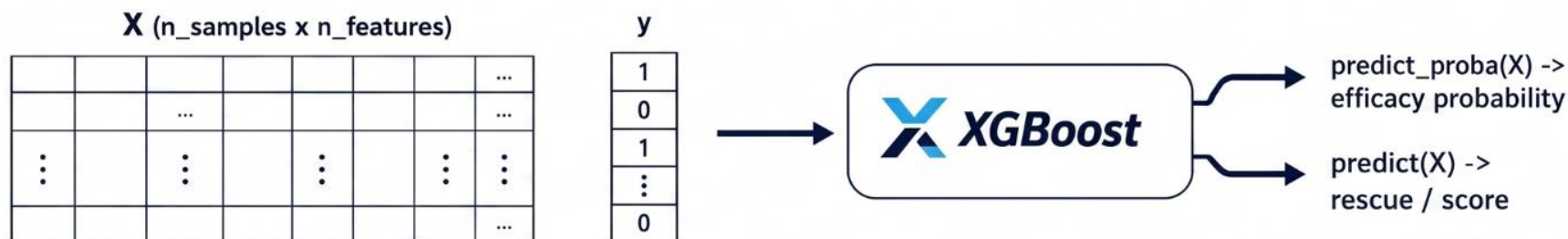
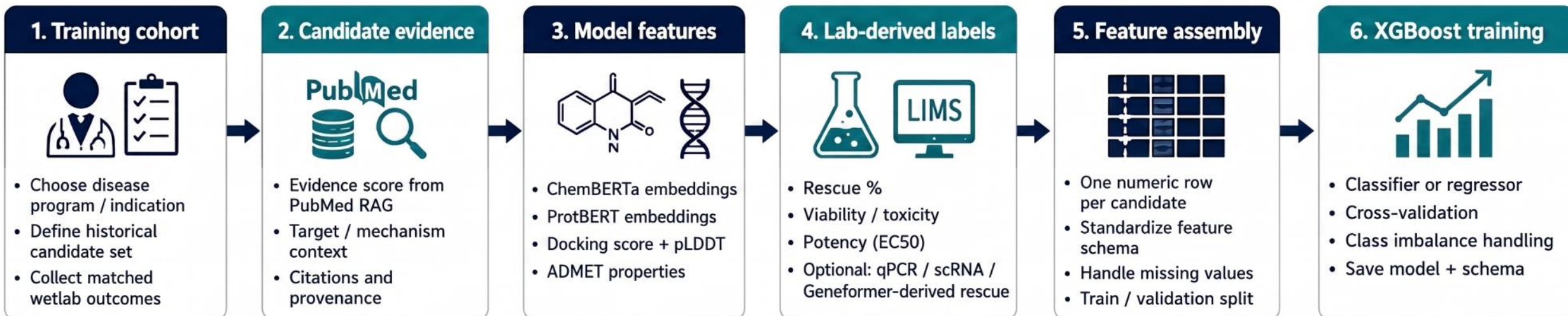
Pretrained models did not directly predict disease efficacy; they generated orthogonal signals that BiotechOS combined into a disease-specific ranking.



Core takeaway: the value was not one model; it was the governed combination of many models into one decision-ready feature pipeline.

XGBoost Training Pipeline — How BiotechOS Learned to Rank Candidates

We used wetlab outcomes as labels and trained a composite ranker on literature, model, structure, and assay-derived features.



Example training row (one candidate)

evidence_score	chemberta_emb_1	chemberta_emb_2	docking_score	pLDDT	solubility	hERG_risk	cyp3e4_inhib	geneformer_rescus	label_y
0.72	-0.341	0.876	-8.70	0.92	-3.10	0.18	0.27	0.64	1

Actual rows contain hundreds to thousands of features.

Saved artifacts

- xgb_ranker.json or .ubj
- feature column order
- training metrics
- data snapshot ID
- version + lineage

Model truth

ChemBERTa / ProtBERT / ESMFold did NOT directly predict efficacy; XGBoost learned how to combine their signals using wetlab labels.

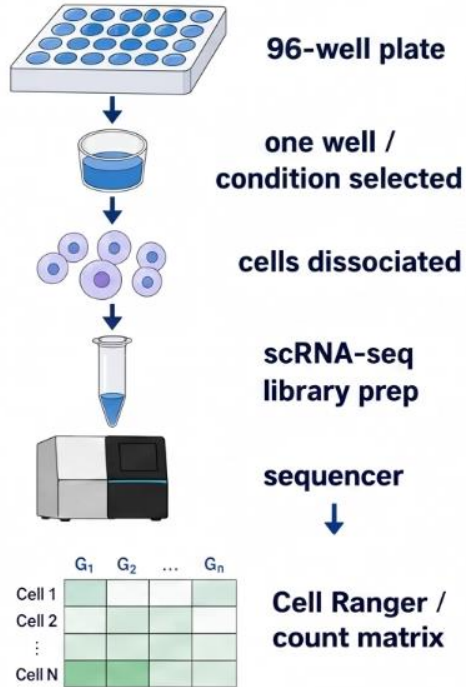


Core takeaway: BiotechOS converted heterogeneous scientific signals into a supervised learning problem, then used XGBoost as the final disease-specific ranker.

scRNA / Geneformer Deep Dive — How Well Data Became Rescue Labels

Ground truth came from expensive transcriptomic measurements; Geneformer turned raw single-cell counts into usable rescue-state signals.

From wetlab to single-cell data



Each cell has barcode + UMI counts across ~20k genes.

plate_id, well_id, condition, dose, timepoint must stay linked.

Geneformer tokenization + model flow

- 1 raw counts saved as .h5ad or .loom
- 2 Geneformer package: **TranscriptomeTokenizer** in geneformer/tokenizer.py
- 3 normalize counts, map genes to Ensembl IDs, rank genes per cell
- 4 convert each cell to an ordered gene-token sequence (up to 4096 tokens)
- 5 Geneformer produces cell embedding / rescue-state signal

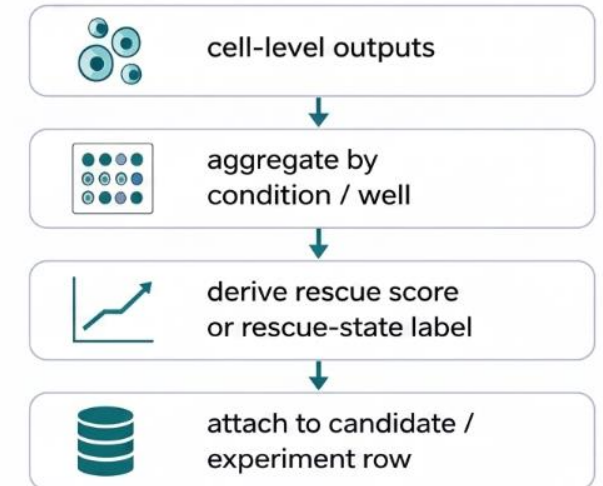
Side note

Geneformer input is not a 4096-value expression vector; it becomes up to 4096 ranked gene tokens per cell.

Illustrative sequence

[DDIT3, HSPA5, XBP1, ...]
↓
[18342, 7719, 14408, ...]

How rescue labels were formed



Why we could not do this on every run

expensive | slower | lower throughput

Why TEER proxy was needed

- 1 match runs between transcriptomic truth and TEER / cheap assays
- 2 build training rows using TEER, O₂, lactate, imaging, qPCR
- 3 use Geneformer-derived rescue as label y
- 4 validate proxy tracks rescue trend, then use broadly

Interview-safe truth



Geneformer itself handled tokenization via TranscriptomeTokenizer; BiotechOS used those truth signals selectively and distilled them into cheaper operational proxies.



Core takeaway: The biology learning loop became economical only after high-fidelity transcriptomic truth was converted into labels that cheaper proxy assays could learn from.

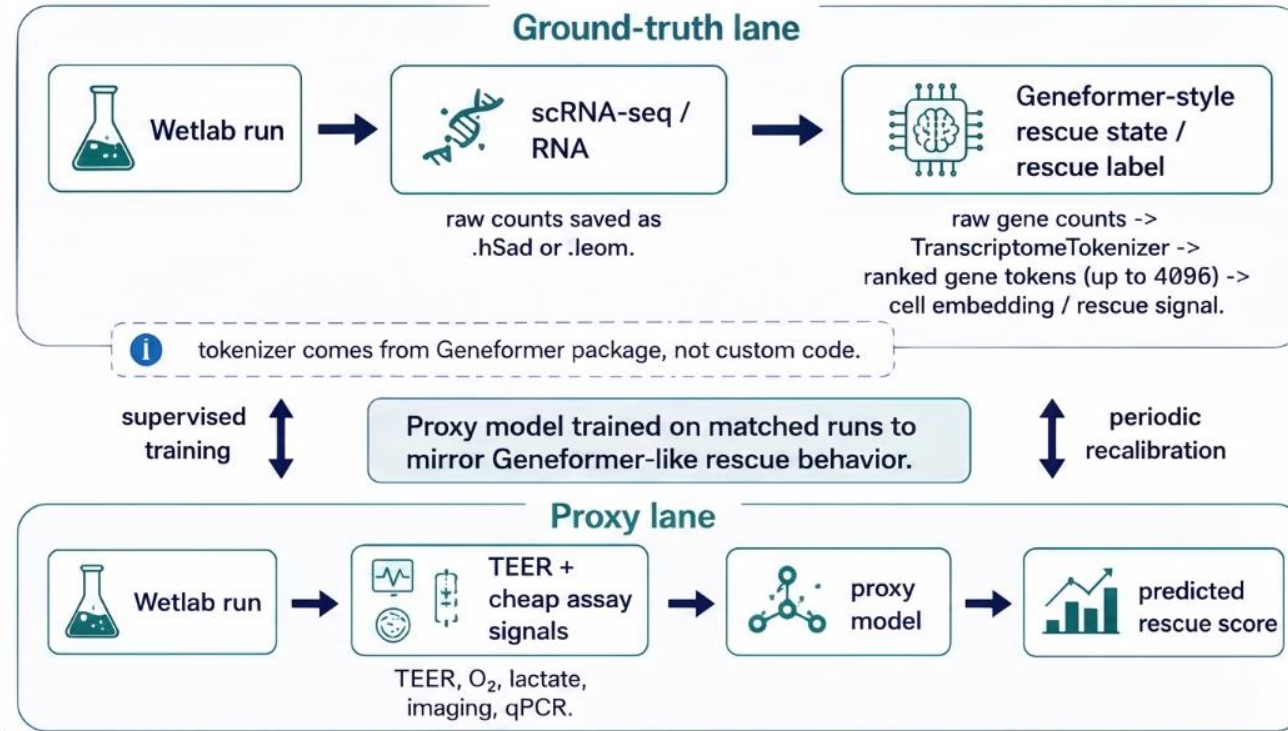
Transcriptomic Truth to TEER Proxy – How We Made Closed-Loop Learning Economical

We used expensive scRNA / Geneformer-style truth selectively, then trained cheaper assay signals to approximate rescue behavior at scale.

Why this was needed

-  scRNA-seq and RNA readouts were rich but expensive
-  cannot run transcriptomics on every candidate / condition
-  drug discovery needs faster, cheaper, repeatable screening
-  goal: preserve biological signal while reducing measurement cost

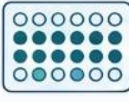
Truth lane vs proxy lane



Operational strategy

-  use scRNA on selected cohorts or calibration runs
-  train proxy on matched runs with both truth and cheap assays
-  use proxy for broad screening
-  periodically re-anchor with transcriptomics
-  reserve expensive truth for edge cases and retraining

Training mechanics

-  Match the same run across plate / well / condition between soRNA truth and TEER features
-  Build training rows using TEER, O₂, lactate, imaging, qPCR as input features
-  Use Geneformer-derived rescue label as y target
-  Validate that proxy predictions track rescue trends before operational use

Biology caveat:
proxy does not replace transcriptomics perfectly; it is a calibrated cost-saving approximation.

Real-World Lab Challenges — Why Biology Was Harder Than the Models

A major part of the work was handling noisy, failure-prone wetlab reality before the AI loop could be trusted.



Assay / physical-world noise

- ⚠ bubbles in wells distorted readouts
- ⚠ temperature drift changed cell behavior
- ⚠ evaporation / edge effects introduced bias
- ⚠ sensor instability and batch effects created noise



Manual -> robotic transition

- manual protocols were not directly machine-executable
- liquid handling constraints changed protocol design
- robot timing / dispense precision mattered
- needed validation before automation could be trusted



Data / biology translation challenges

- ⚠ RNA and scRNA outputs were expensive and heterogeneous
- ⚠ mapping plate / well / sample / condition had to be exact
- ⚠ comparing runs across assays required normalization
- ⚠ Geneformer-grade signals were rich but not cheap enough for everything



Operational ML challenges

- lab labels were noisy and delayed
- schema drift broke downstream pipelines
- retraining required version discipline
- confidence and human review were needed for low-signal cases

How we handled it



QC flags and quarantine



metadata capture on every run



human approval gates



fallback / retry paths



periodic recalibration



exclude or down-weight bad runs



Detect noise



Net resolution

capture metadata



use only trustworthy data for training



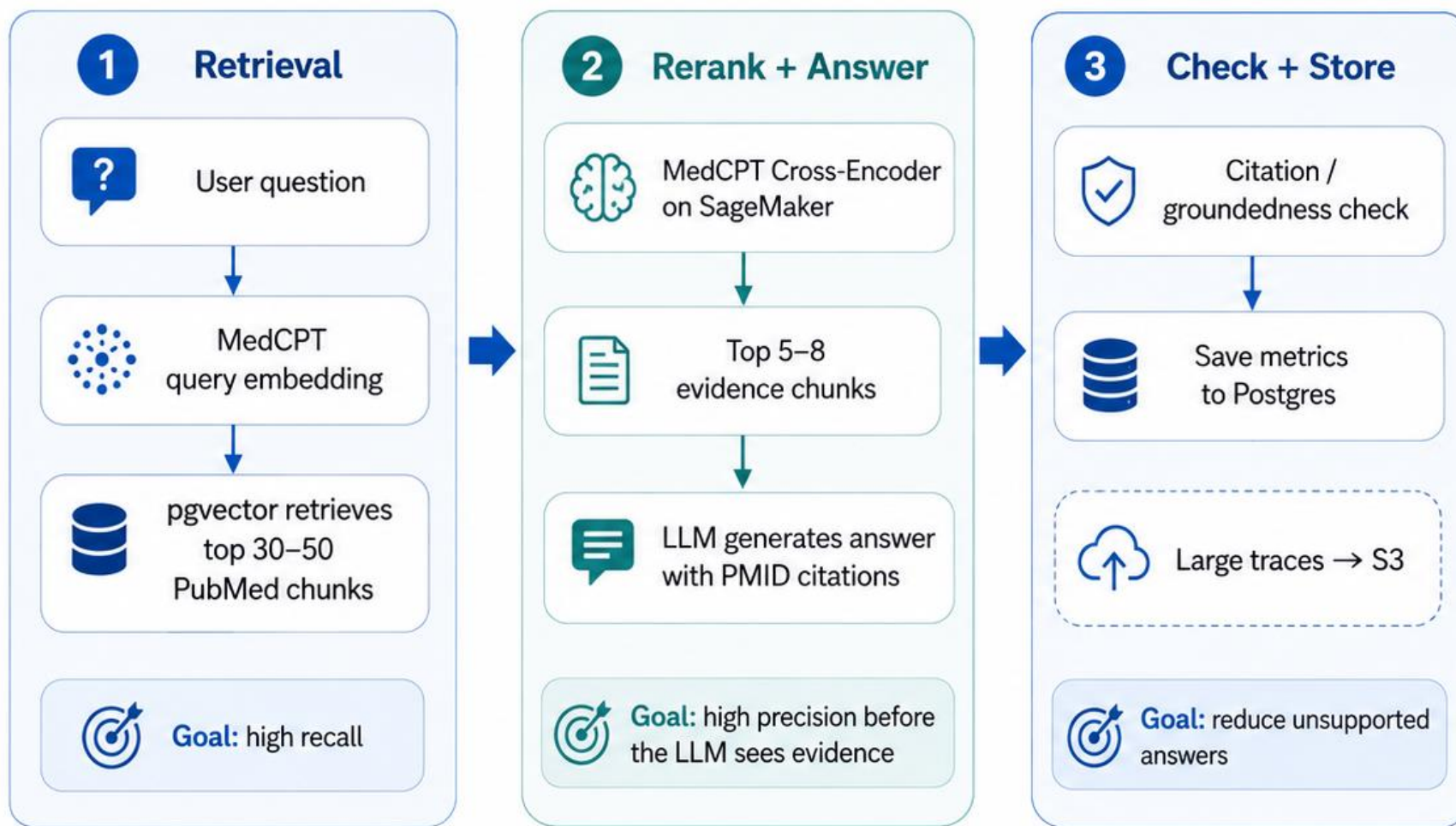
Biology was not like software; the system had to tolerate messy experiments, not just clean API calls.



Core takeaway: the hard problem was not only ranking molecules; it was making unreliable biological operations reliable enough to learn from.

RAG Quality Controls

Simple, defensible quality gates for PubMed question answering



Core metrics and what they mean



Recall@50

Did retrieval bring back the known relevant PubMed paper in the top 50?



Precision@5

Of the top 5 reranked chunks, how many were truly useful evidence?



Citation coverage

What % of the final answer is backed by cited PubMed chunks?



Groundedness pass rate

What % of answers are fully supported by retrieved evidence?



No-answer rate

How often the system abstains because evidence is weak or missing.



Quality is measured on a curated golden set; live traffic is monitored with proxy signals such as low retrieval score, low reranker score, latency, and error rate.

Production Safety & Failure Handling

Simple, defensible protections for login, prompt safety, model failures, and user experience

1) Login & Access



- Amazon Cognito / OIDC login
- MFA for admins
- JWT validated by FastAPI
- Tenant/project RBAC on every API call
- Audit trail for jobs, prompts, and model versions

2) Prompt Injection Protection



- Retrieved PubMed text is treated as evidence, not instructions
- No tool calls are allowed from retrieved documents
- Ranking jobs require validated JSON schema
- Unsupported or unsafe requests are refused

3) Model Failure Handling



- 🕒 Timeouts
- ↓ Retries with exponential backoff
- 🔄 Circuit breaker per SageMaker endpoint
- 📧 Dead-letter queue for repeated failures
- 🖥️ Health checks + CloudWatch alarms

i ChemBERTa, ProtBERT, ESMFold, ADMET, XGBoost are SageMaker endpoints.

4) User / App Flow on Failure



- A PubMed Q&A**
 - If reranker fails, fall back to pgvector-only retrieval and show lower-confidence banner
- B Molecule ranking job**
 - Show job states: queued → retrying → degraded → failed / complete
 - Never fabricate missing model outputs; surface clear status in chat



Principle: Fail visibly, preserve audit trail, and never generate scientific conclusions from missing evidence.



Stored in Postgres:
job status, endpoint status,
failure reason, retry count



Stored in S3:
large traces